

Sistemas Compostos de IA: Otimização de Respostas em Benchmarks Objetivos por meio de Orquestração Baseada no Ralph Loop

Hugo Emílio Nomura, Pedro Henrique Correia de Oliveira, Pedro Henrique Satoru Lima Takahashi, Vitor Monteiro Vianna

Ciência da Computação – Centro Universitário FEI, campus São Paulo

unifhnomura@fei.edu.br, unifpoliveira@fei.edu.br, unifptakahashi@fei.edu.br, unievvianna@fei.edu.br

Orientador: Charles Henrique Porto Ferreira

Departamento de Ciência da Computação, Centro Universitário FEI

cferreira@fei.edu.br

Resumo: Os Modelos de Linguagem de Grande Porte (LLMs) evoluíram de geradores de texto para motores centrais de raciocínio em tarefas complexas. Contudo, arquiteturas agentais tradicionais frequentemente processam múltiplas etapas acumulando histórico em uma única janela, o que leva à degradação de desempenho conhecida como *context rot* (esgotamento de contexto). Para contornar essa limitação estrutural, este trabalho propõe e avalia uma arquitetura de orquestração cognitiva estrita (*harness*), inspirada no paradigma *stateless* do *Ralph Wiggum Loop*. Em contraste com as inferências monolíticas e lineares, a abordagem isola a execução em fases discretas de análise, decomposição e validação iterativa, reiniciando o contexto entre os passos para mitigar alucinações e omissões lógicas. Para validar empiricamente a eficácia da abordagem, estabelecemos um plano de avaliação comparativo utilizando modelos de diferentes escalas de parâmetros (como a família Llama 3.1, Claude 3.5 Sonnet e GPT-4o). O estudo utiliza bases de dados acadêmicas rigorosas de propósito geral — notadamente MMLU, GPQA e MMLU-Pro — para investigar a hipótese de que a orquestração estruturada do raciocínio e o isolamento de contexto são capazes de elevar sistematicamente a acurácia, independentemente da escala inerente do modelo.

Palavras-chaves: Esperar ate o fim do desenvolvimento do glossario, e deposi colocar aqui

I. Introdução

Os modelos de linguagem de grande porte (*Large Language Models* — LLMs) deixaram de ser meramente geradores de texto e hoje funcionam como plataformas compostas por camadas de processamento, coordenação e ferramentas auxiliares. Essa transição viabiliza capacidades como raciocínio em múltiplas etapas, orquestração de subagentes e uso de ferramentas externas, porém também revela limitações estruturais: nem sempre a melhoria observada no comportamento final advém apenas do modelo base, mas das arquiteturas que organizam e abstraem o processo de inferência.

Este trabalho parte de dois problemas centrais observados na prática e documentados na literatura: (1) a aparente discórdância entre respostas iniciais e respostas revisadas por um mesmo modelo, e (2) a forma como fluxos de pensamento (prompting encadeado, agentes e pipelines) são atualmente implementados, incluindo suas fragilidades frente ao crescimento do contexto e esgotamento da janela de atenção [1].

Técnicas consolidadas na literatura, como *Chain-of-Thought* [2], *ReAct* [3] e abordagens de autorreflexão [4], já demonstraram que processos iterativos superam as inferências de passo único. Contudo, em arquiteturas populares de agentes, o modelo tende a operar acumulando um histórico crescente de raciocínios em uma única janela. Isso frequentemente resulta em alucinações e degradação devido à compactação excessiva do contexto. Para contornar essa

limitação, propomos a estruturação do raciocínio baseada em princípios de decomposição e *statelessness* aplicados de forma determinística.

Neste trabalho, adotamos a hipótese de que a qualidade objetiva das saídas em tarefas complexas depende tanto do LLM base quanto do “*Harness*” que governa seu uso. Para testar essa premissa, nossa meta é avaliar o impacto da orquestração estruturada — inspirada no *Ralph Wiggum Loop* [5] — frente à inferência direta tradicional. A avaliação em andamento baseia-se em bases de dados (*datasets*) acadêmicas rigorosas, visando demonstrar se uma arquitetura composta por etapas de decomposição e verificação iterativa é capaz de elevar o desempenho de modelos com menos parâmetros a patamares competitivos, superando abordagens lineares em métricas de acurácia.

Em suma, as principais contribuições almejadas por este estudo são:

- **Sistematização do paradigma *stateless*:** Implementação de um pipeline de inferência que isola o contexto entre etapas, mitigando alucinações ligadas ao histórico longo.
- **Adaptação de Domínio:** Aplicação do *Ralph Wiggum Loop* — originado na automação de engenharia de software — para resolução estruturada de problemas de propósito geral e conhecimento enciclopédico.
- **Análise Quantitativa Estratificada:** Estabelecimento de um *benchmark* cruzado avaliando o impacto

da orquestração sobre a inferência linear, utilizando modelos abertos (Llama 3.1) e fechados (GPT-4o, Claude 3.5 Sonnet).

II. Trabalhos Relacionados

O desenvolvimento de arquiteturas e técnicas de *har-ness* para potencializar o raciocínio de LLMs tem sido um campo ativo de pesquisa, subdividindo-se em estratégias de *prompting* estendido e fluxos de trabalho baseados em agentes.

A. Estratégias de Raciocínio e Agentes

A literatura demonstra amplamente que a inferência primária de um modelo pode ser significativamente aprimorada quando estruturada em fases. O método *Chain-of-Thought* [2] e sua generalização *Tree of Thoughts* [6] evidenciaram a importância de induzir o modelo a explicitar passos intermediários. Abordagens como *ReAct* [3] deram um passo além ao sinergizar o raciocínio interno com a capacidade de invocar ferramentas externas (*Acting*), permitindo contornar alucinações por meio de observações diretas do ambiente.

Paralelamente, pipelines interativos baseados em *feed-back* demonstram alta eficácia. *Reflexion* [4] implementou ciclos em que o modelo atua como seu próprio crítico, revisando saídas com base em reforço verbal externo, e o *Self-Debugging* [7] mostrou que os LLMs podem depurar seus próprios erros quando guiados de forma sistemática. Adicionalmente, métodos focados na redução da sobrecarga cognitiva através da divisão de tarefas complexas — como *Least-to-Most* [8], *Decomposed Prompting* [9] e *Plan-and-Solve* [10] — sustentam a premissa de que separar planejamento e execução mitiga falhas lógicas e omissões.

B. Orquestração Estruturada e o Ralph Wiggum Loop

Embora as técnicas supracitadas representem a base teórica de agentes autônomos, muitas dependem do acúmulo contínuo de estado em uma única janela de contexto, fenômeno que agrava a degradação e o esquecimento descritos em *Lost in the Middle* [1]. Frente a esse problema, o *Ralph Wiggum Loop* [5] emergiu na comunidade de engenharia de software como um paradigma pragmático alternativo. A técnica isola a execução em etapas rigidamente separadas e reinicializa o contexto a cada iteração (*Fresh Context*), armazenando estritamente os resultados intermediários essenciais.

A validação acadêmica rigorosa de uma estrutura semelhante a esta foi recentemente evidenciada por McComb et al. [11], que avaliaram o impacto de agentes baseados no *Ralph Wiggum Loop* para a resolução de problemas complexos de design de pacotes de baterias na engenharia. O estudo comprovou não apenas a eficácia da abordagem *stateless*, mas propôs aprimoramentos através de ciclos metacognitivos de co-regulação (CRDAL), nos quais um agente avaliador atua criticando ativamente o design a fim de mitigar vieses e fixações de paradigma.

Este trabalho apoia-se em fundamentações similares para justificar o uso do paradigma de execução *stateless*

em nossa orquestração. Contudo, enquanto McComb et al. [11] concentraram a arquitetura na otimização de métricas tangíveis de engenharia e modelagem paramétrica, nossa metodologia isola e adapta os ganhos da orquestração estrita do *Ralph Loop* para elevar a capacidade de resolução de questões de propósito geral em múltiplos domínios do conhecimento (representados pelos *datasets* MMLU e GPQA).

III. Metodologia

A. Análise macro do processo de thinking

O diagrama a seguir apresenta o fluxo macro do pipeline de *thinking* proposto neste trabalho, oferecendo uma visão consolidada de todas as fases que o compõem.

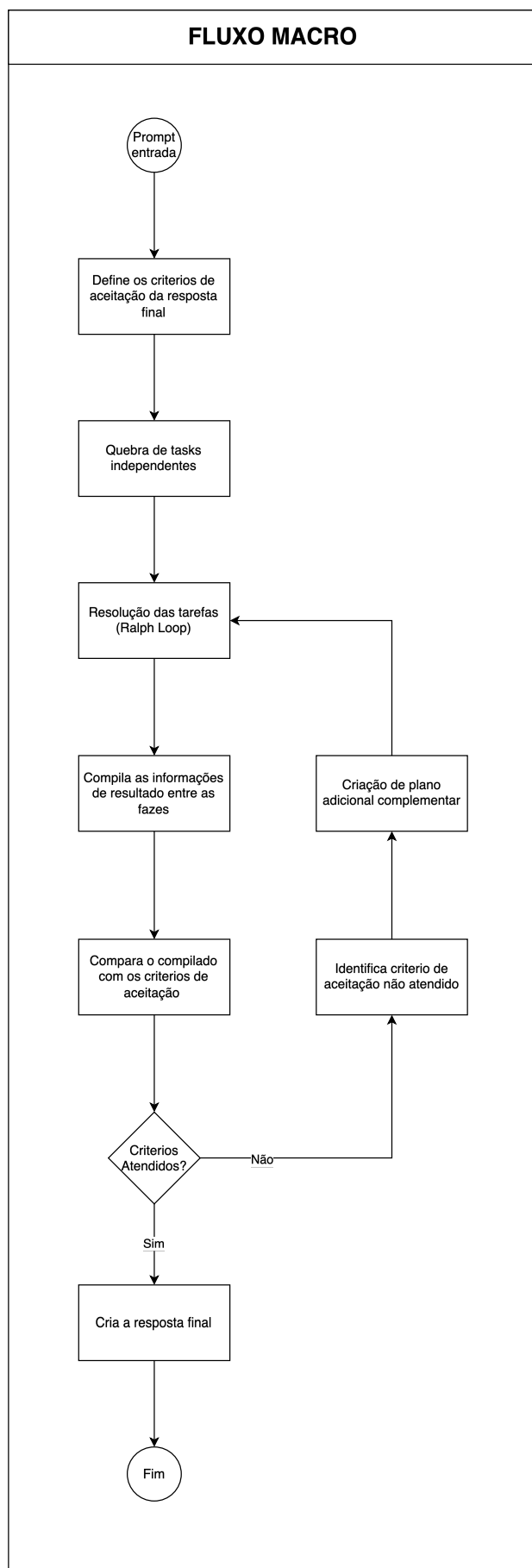


Figura 1. Fluxo macro do pipeline de *thinking*.

O pipeline é iniciado a partir de um *prompt* de entrada fornecido pelo usuário. A partir desse insumo, o processo percorre as seguintes etapas em sequência: (i) definição dos critérios de aceitação da resposta final e do formato de saída esperado; (ii) decomposição do problema em tarefas atômicas e independentes; (iii) resolução iterativa de cada tarefa por meio de um ciclo de execução e verificação inspirado no *Ralph Wiggum Loop*; (iv) avaliação dos resultados consolidados frente aos critérios de aceitação estabelecidos; e, por fim, (v) composição da resposta final ao usuário. Caso a avaliação dos critérios indique que nem todos foram atendidos, o pipeline não descarta o trabalho realizado — em vez disso, cria um plano adicional complementar voltado exclusivamente para cobrir as lacunas identificadas, retornando à fase de resolução de tarefas. Esse comportamento confere ao sistema um caráter iterativo e convergente.

Um aspecto central da arquitetura é a separação deliberada entre as fases de execução: cada etapa opera de forma *stateless* em relação às demais, no sentido de que não carrega diretamente o histórico de raciocínio das fases anteriores. Em vez disso, o sistema preserva apenas artefatos estruturados — em especial os arquivos `planning.json` e `tasks_results.json` — que funcionam como memória persistente e controlada entre as etapas. Essa decisão de projeto tem dois objetivos complementares: por um lado, mitigar os problemas de degradação discutidos na Introdução, nos quais o crescimento progressivo do contexto leva ao fenômeno frequentemente referido como *context rot* — o acúmulo de informação irrelevante, resumos implícitos e ruído que diluem o conteúdo útil e prejudicam a qualidade das respostas —; por outro, garantir que cada chamada ao LLM opere com o menor contexto possível, reduzindo o risco de que informações e raciocínios de tarefas anteriores contaminem a execução da tarefa corrente, introduzindo inconsistências ou “lixo de contexto” nas respostas geradas.

B. Análise micro das fases

A seguir, cada fase do pipeline é detalhada individualmente. Para cada fase, são descritos: os agentes de LLM envolvidos, os dados de entrada e saída, e o fluxo de controle interno. Essa granularidade tem o objetivo de tornar o processo inteiramente replicável por pesquisadores externos.

B.1 Agentes de LLM

Neste trabalho, adota-se uma definição restrita de *agente*: trata-se de um componente composto exclusivamente por um *prompt* pré-definido e uma chamada ao modelo de linguagem (*LLM_CALL*). Diferentemente de arquiteturas que atribuem a agentes acesso a ferramentas externas, chamadas a APIs, servidores MCP ou capacidades de navegação, os agentes aqui descritos operam de forma isolada — recebem um contexto de entrada estruturado, processam via LLM e retornam uma saída textual ou estruturada em JSON. Essa restrição é intencional: ao eliminar dependências externas, torna-se possível avaliar de forma mais direta o impacto da orquestração e do *prompting* sobre a qualidade das respostas, isolando as variáveis de interesse

do estudo.

Todos os *prompts* de agentes são redigidos em inglês. Estudos como o de Huang et al. [12] e Shi et al. [13] mostram que modelos de linguagem multilíngues tendem a produzir resultados de maior qualidade quando instruídos no idioma em que foram mais amplamente treinados — predominantemente o inglês —, independentemente do idioma da questão de entrada. Qin et al. [14] reforçam esse achado ao demonstrar que a combinação de *prompting* em inglês com raciocínio *zero-shot* em cadeia produz ganhos consistentes mesmo em línguas de menor representação nos dados de treinamento. A adoção do inglês como idioma padrão dos *prompts* de agentes visa, portanto, maximizar a qualidade do raciocínio interno do modelo, sem prejuízo ao idioma da resposta entregue ao usuário — que é determinado separadamente, conforme descrito na seção seguinte.

Adicionalmente, todos os *prompts* são construídos utilizando a sintaxe *Markdown*, com delimitadores explícitos de seção para separar instruções gerais, contexto de entrada e especificações de saída. Estudos como o de He et al. [15] e Liu et al. [16] mostram que a formatação do *prompt* afeta de forma mensurável o desempenho dos modelos, e que otimizações conjuntas de conteúdo e formato produzem ganhos adicionais sobre a otimização exclusiva do conteúdo textual. A adoção de *Markdown* visa, portanto, tornar delimitadores, hierarquias e restrições mais salientes durante a inferência, contribuindo para respostas mais consistentes com as instruções fornecidas.

B.2 Definição dos critérios de aceite da resposta final

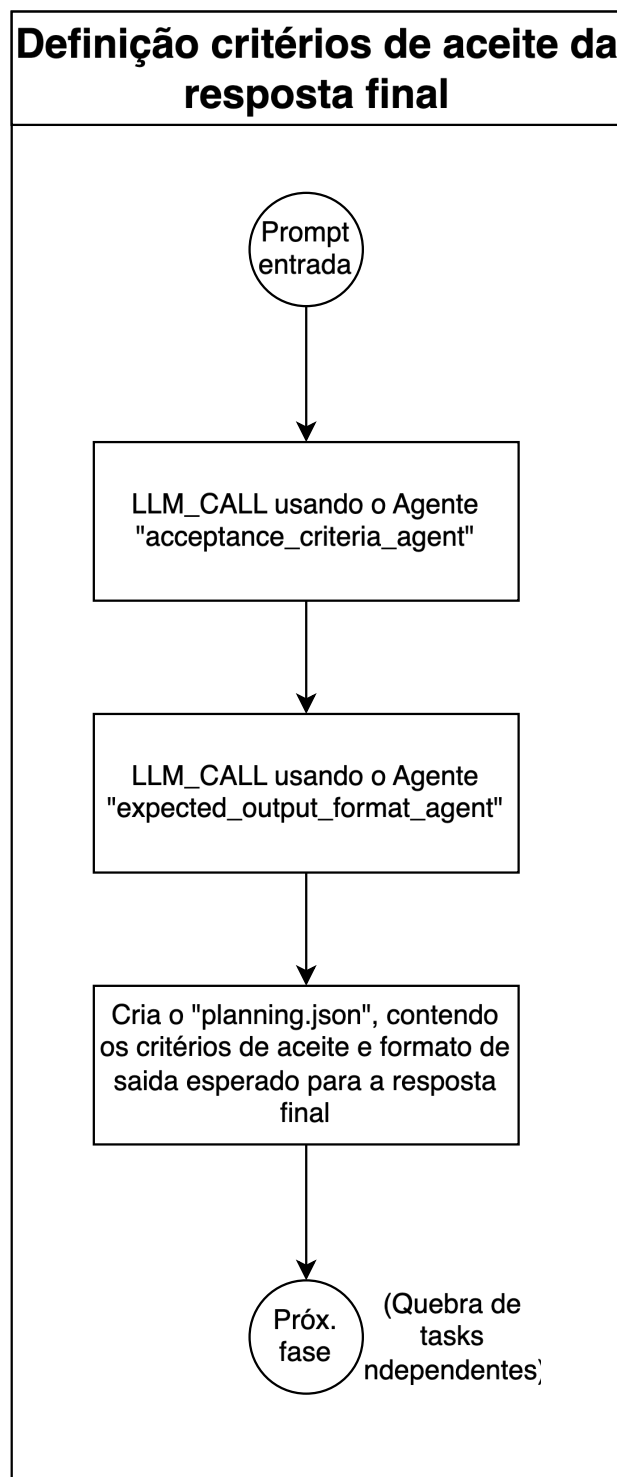


Figura 2. Fase de definição dos critérios de aceite da resposta final.

Esta fase é responsável por transformar o *prompt* bruto do usuário em dois artefatos que guiarão todo o processo posterior: (i) uma lista de critérios de aceitação que a resposta final deverá satisfazer, e (ii) uma especificação do formato de saída esperado. Para isso, dois agentes são acionados em sequência a partir da mesma entrada.

O `acceptance_criteria_agent` recebe o *prompt* de entrada do usuário e é responsável por identificar e listar, de forma explícita, os critérios objetivos e verificáveis que uma resposta satisfatória deve cumprir. Em seguida, o `expected_output_format_agent` analisa o mesmo *prompt*, buscando por indicações explícitas de formato de saída — como “retorne somente a alternativa correta”, “responda em JSON” ou “elabore uma justificativa”. Caso nenhuma indicação seja encontrada, o agente infere o formato mais adequado ao tipo de tarefa: para questões de múltipla escolha, o padrão é “retornar somente a letra da alternativa correta, sem explicações adicionais”; para questões abertas, “texto corrido em formato Markdown”. O idioma da resposta final é tratado como parte integrante do formato de saída: se o usuário o especificou explicitamente, essa instrução é acatada; caso contrário, o agente infere o idioma a partir do idioma utilizado na entrada e o registra como especificação. Quando exemplos de saída são fornecidos pelo usuário, estes são incorporados ao campo correspondente.

Os resultados dessas duas chamadas são então consolidados em um arquivo JSON denominado `planning.json`, que servirá de artefato central de memória estruturada para todas as fases subsequentes. A estrutura do `planning.json` é inspirada no `prd.json` do *Ralph Wiggum Loop* original, porém estendida para incorporar as informações de critérios de aceitação e formato de saída.

```
{
  "general_infos": {
    "acceptance_criteria": [],
    "expected_output_format": {
      "format": "",
      "response_language": ""
    }
  },
  "tasks": []
}
```

O campo `tasks` é inicialmente vazio e será preenchido pela fase seguinte. A separação entre `general_infos` e `tasks` é proposital: as informações gerais são invariantes ao longo de todo o pipeline, enquanto as tarefas podem ser adicionadas iterativamente por planos complementares.

B.3 Quebra de Tasks Independentes

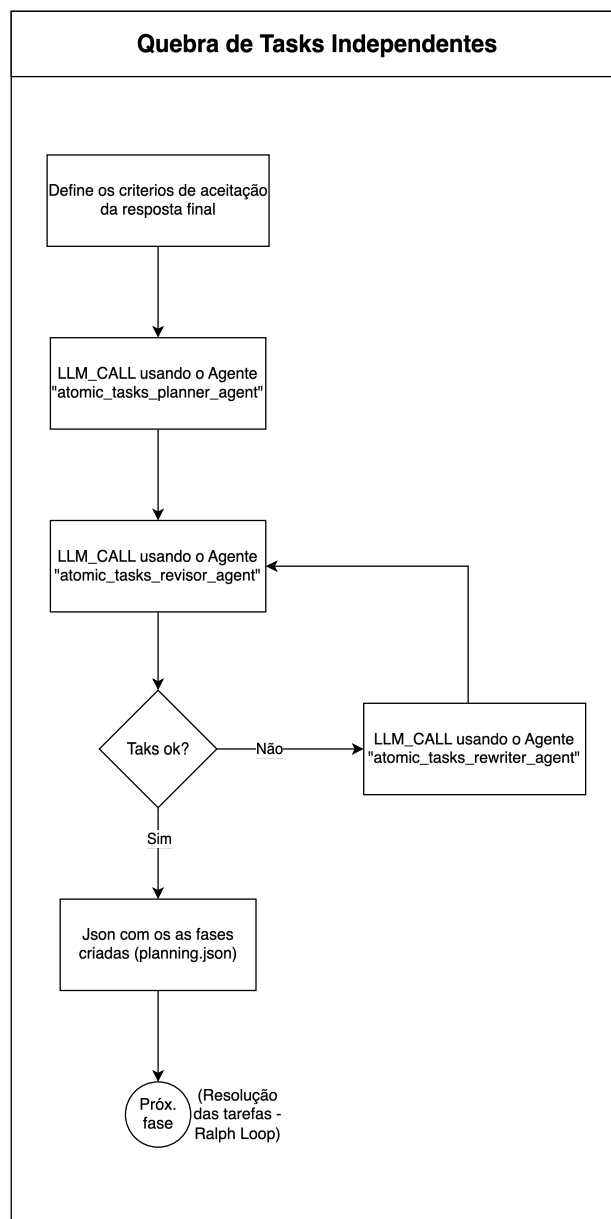


Figura 3. Fase de quebra de tarefas independentes.

Esta fase é, em termos de responsabilidade lógica, a mais crítica de todo o pipeline: é aqui que se implementa o princípio central do *Ralph Wiggum Loop* — a decomposição do problema em unidades mínimas e independentes de execução. O agente `atomic_tasks_planner_agent` recebe como entrada o *prompt* do usuário e o campo `general_infos` do `planning.json` gerado na fase anterior, e a partir disso constrói um plano de tarefas atômicas.

O conceito de atomicidade empregado é preciso: uma tarefa é considerada atômica quando representa a menor unidade de trabalho possível e pode ser executada e avaliada de forma completamente independente das demais tarefas do plano, sem necessidade de acessar ou reutilizar os resultados de outras tarefas durante sua própria execução.

Essa propriedade é fundamental para garantir o funcionamento correto do mecanismo de *fresh context* — se tarefas dependessem umas das outras durante a execução, seria necessário carregar contexto acumulado, o que reintroduziria os problemas de degradação que a arquitetura busca mitigar.

Após a geração do plano inicial, o agente `atomic_tasks_revisor_agent` é acionado para verificar se todas as tarefas propostas satisfazem o critério de independência, retornando um JSON com um campo booleano de aprovação e uma justificativa. Se problemas forem identificados, o agente `atomic_tasks_rewriter_agent` — que incorpora em seu *prompt* todas as regras e restrições do planejador — é acionado para reescrever o plano com base na justificativa fornecida. Esse ciclo de revisão repete-se até que o revisor valide o plano, garantindo que apenas tarefas verdadeiramente independentes avancem para a fase de execução.

Cada tarefa do plano é representada no `planning.json` no seguinte formato:

```
{
  "id": "",
  "title": "",
  "description": "",
  "acceptance_criteria": [],
  "pass_phase": false
}
```

O campo `pass_phase` é inicializado como `false` para todas as tarefas e atualizado para `true` ao final da execução bem-sucedida de cada uma, funcionando como marcador de progresso para o mecanismo iterativo.

B.4 Resolução das tarefas — Ralph Loop

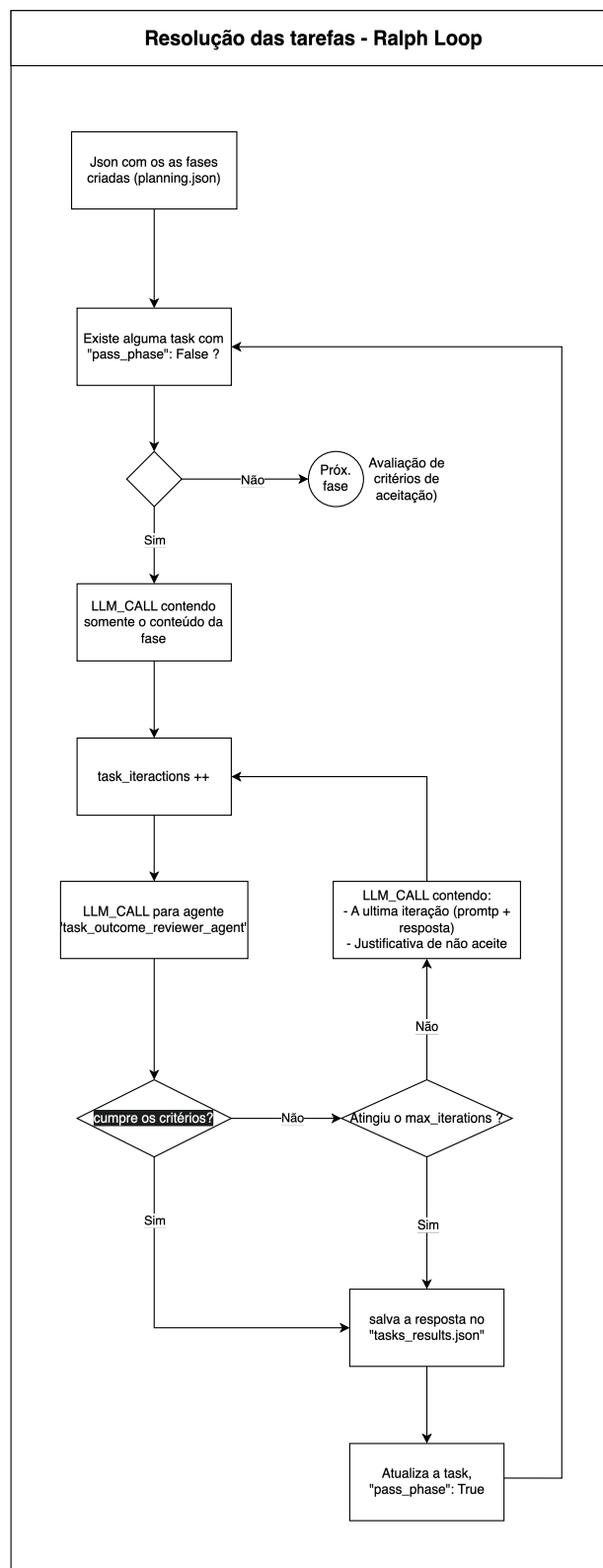


Figura 4. Resolução iterativa das tarefas utilizando o Ralph Loop.

Esta fase implementa o núcleo iterativo do *Ralph Wiggum Loop* adaptado ao contexto deste trabalho. O

processo inicia pela verificação do `planning.json`: enquanto existirem tarefas com `pass_phase: false`, o pipeline seleciona a próxima tarefa pendente e inicia um ciclo de tentativa–avaliação–revisão.

Cada iteração do ciclo consiste em: (i) uma chamada ao LLM contendo exclusivamente o conteúdo da tarefa atual — sem histórico acumulado de outras tarefas —, seguida de (ii) uma chamada ao agente `task_outcome_reviewer_agent`, que compara a resposta gerada com os `acceptance_criteria` da tarefa. O revisor retorna um JSON com um campo booleano de aprovação e uma justificativa. Se o revisor aprovar a resposta, ela é salva no arquivo `tasks_results.json` e o campo `pass_phase` da tarefa é atualizado para `true`. Se reprovar, a justificativa é incorporada à próxima chamada ao LLM, configurando um ciclo de refinamento guiado, análogo aos mecanismos de *Reflexion* [4] e *Self-Debugging* [7].

No que diz respeito à persistência dos resultados, adota-se uma adaptação do princípio original do *Ralph Wiggum Loop*, que preconiza que “All memory lives in files and git — not the model”. Como o presente trabalho aplica o pipeline a cenários distintos do desenvolvimento de software, o mecanismo de *commit* em repositório Git é substituído pelo arquivo `tasks_results.json`, que acumula exclusivamente as respostas finais aprovadas de cada tarefa — sem registrar histórico de raciocínio, tentativas anteriores ou qualquer metadado do ciclo iterativo.

Para mitigar o risco de *deadlock*, implementa-se um mecanismo de controle denominado *pânico*. Um contador `task_interactions` é incrementado a cada chamada ao LLM dentro do ciclo de uma tarefa. Quando esse contador atinge o valor de `max_iterations`, a fase encerra automaticamente e a resposta disponível no momento é salva no `tasks_results.json`, mesmo que os critérios não tenham sido plenamente atendidos.

B.5 Avaliação dos critérios de aceitação

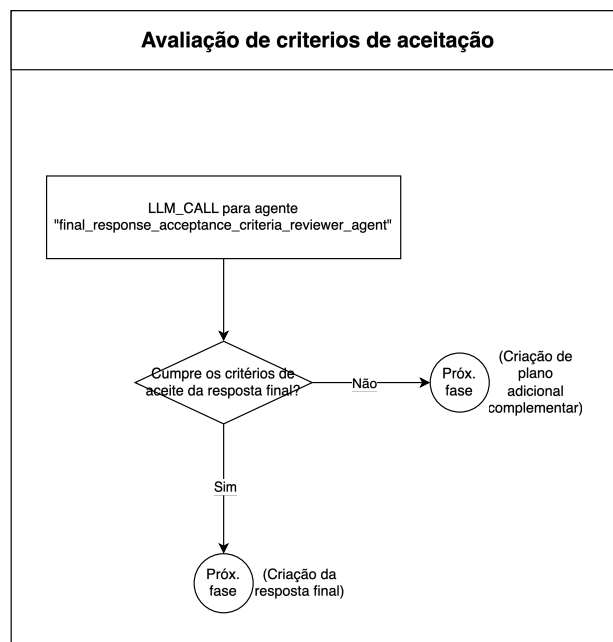


Figura 5. Avaliação global dos critérios de aceitação.

Ao término da resolução de todas as tarefas do plano, o pipeline consolida os resultados armazenados no `tasks_results.json` e os submete a uma avaliação global frente aos critérios de aceitação definidos em `general_infos.acceptance_criteria` do `planning.json`. Essa fase é executada pelo agente `final_response_acceptance_criteria_reviewer_agent` que recebe ambos os artefatos como entrada.

A lógica é análoga à dos revisores de tarefa individual, porém operando em um nível de abstração superior: em vez de avaliar uma resposta pontual contra critérios locais de uma tarefa, o agente avalia se o conjunto total de resultados gerados fornece insumos suficientes para construir uma resposta final que atenda a todos os critérios globais. Dois fluxos de saída são possíveis a partir dessa avaliação: caso todos os critérios sejam considerados atendidos, o pipeline avança para a criação da resposta final; caso contrário, avança para a criação de um plano adicional complementar.

B.6 Criação de plano complementar

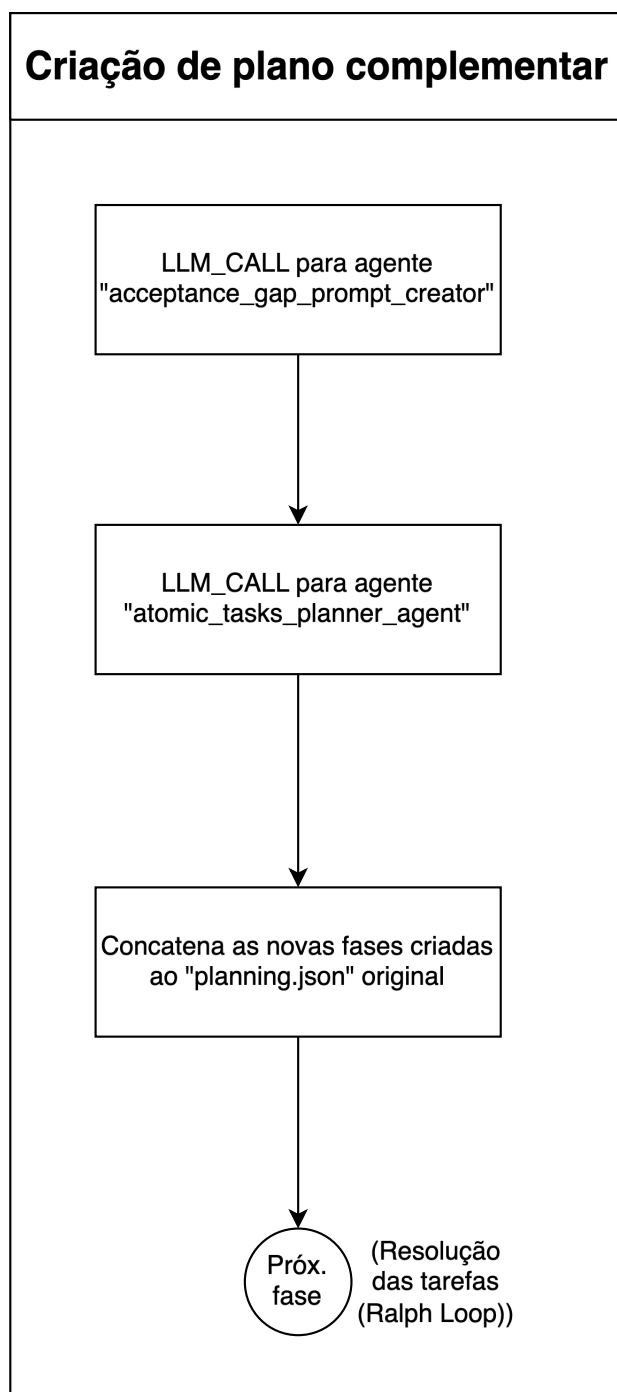


Figura 6. Criação de plano complementar para critérios não atendidos.

Quando a avaliação global indica que os critérios de aceitação não foram plenamente atendidos, o pipeline não retorna ao início — o que implicaria descartar e reprocessar etapas já concluídas. Em vez disso, adota-se uma estratégia de afunilamento progressivo: o agente `acceptance_gap_prompt_creator` analisa a justificativa retornada pelo revisor global e produz um novo *prompt* no estilo de entrada de usuário, descrevendo exclu-

sivamente o que ainda precisa ser resolvido para cobrir as lacunas identificadas.

Esse *prompt* é então submetido ao `atomic_tasks_planner_agent`, o mesmo agente da fase de quebra de tarefas independentes, que gera um novo conjunto de tarefas complementares. Essas novas tarefas são concatenadas ao campo `tasks` do `planning.json` original, e o pipeline retorna à fase de resolução das tarefas para executá-las.

Esse mecanismo constitui, na prática, um segundo nível de loop que garante a convergência iterativa do pipeline: a cada ciclo, o conteúdo gerado aproxima-se do que é necessário para satisfazer todos os critérios de aceitação, sem desperdiçar o trabalho já realizado nas iterações anteriores.

B.7 Criação da resposta final

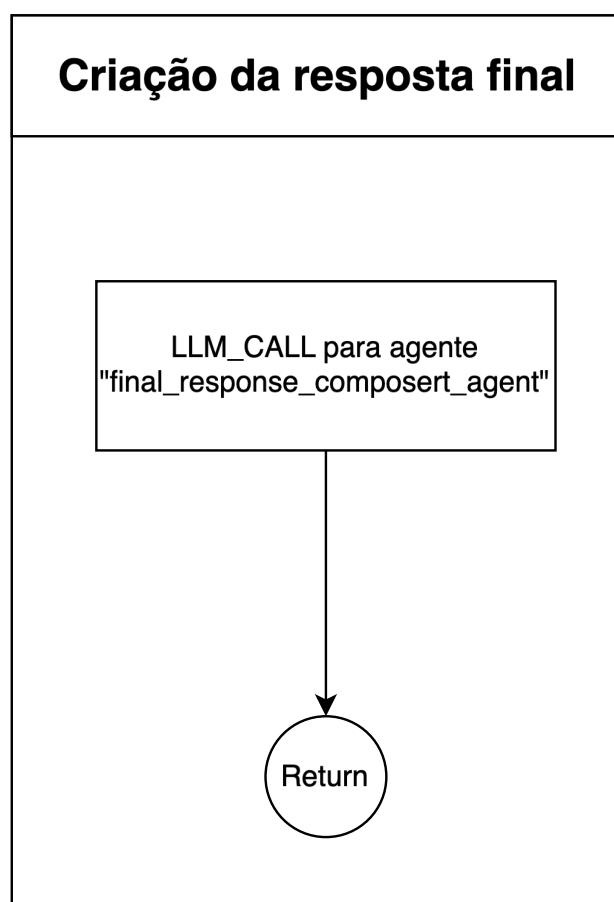


Figura 7. Fase de composição da resposta final.

A fase de criação da resposta final assume que o `tasks_results.json` contém todos os insumos necessários para compor uma resposta que satisfaça integralmente os critérios de aceitação globais — premissa garantida pela fase de avaliação anterior. É nesta fase, e somente nesta, que o conteúdo do `tasks_results.json` é efetivamente recuperado e utilizado.

O agente `final_response_composer_agent` recebe como entrada o arquivo

completo de resultados, o campo `general_infos.expected_output_format` do `planning.json` e o `prompt` original do usuário, utilizando-os em conjunto para compor a resposta final no formato e idioma especificados. O retorno desse agente é a saída direta do pipeline ao usuário, encerrando o processo de *thinking*.

IV. Plano de avaliação

O plano de avaliação visa mensurar o impacto da arquitetura de orquestração na acurácia de modelos de linguagem em tarefas de raciocínio geral, comparando sistematicamente a inferência direta com a orquestrada. Para essa validação, a metodologia contempla cinco modelos selecionados para cobrir uma progressão de complexidade: Llama 3.1 (8B, 70B e 405B), Claude Sonnet 4.6 e GPT 5.4.

Para cada modelo, está prevista a comparação de duas configurações: (i) uma inferência "linear", realizada por meio de uma requisição direta ao modelo sem processamento auxiliar, e (ii) a mesma inferência mediada pelo *Harness* proposto, empregando ciclos de verificação e ferramentas externas. O desempenho será medido por meio de 180 questões amostradas das bases GPQA [17], MMLU [18] e MMLU-Pro [19], escolhidas por seu elevado rigor acadêmico. A métrica primária consistirá na acurácia, obtida pela correspondência estrita entre a alternativa final selecionada pelo sistema e o gabarito.

A execução seguirá um protocolo no qual, inicialmente, os modelos estabelecem seu desempenho de base na inferência padrão. Na sequência, as métricas serão reavaliadas sob a arquitetura orquestrada (inicialmente focada na família Llama), sempre respeitando restrições operacionais *stateless*. O objetivo esperado desta avaliação é verificar se o *harness* altera o posicionamento dos modelos no ranking global, testando a hipótese de que a orquestração estruturada suplanta ganhos atrelados puramente à escala de parâmetros.

A Tabela I apresenta as projeções de acurácia que fundamentam este plano e o benchmark pretendido para a validação da arquitetura.

Tabela I. Projeção de acurácia média esperada por modelo e configuração (dados hipotéticos para fins de planejamento).

Modelo	Configuração	Acurácia Esperada
Llama 3.1 8B	Direto	52%
Llama 3.1 70B	Direto	74%
Llama 3.1 8B	Ralph Loop	77%
Llama 3.1 405B	Direto	84%
Llama 3.1 70B	Ralph Loop	88%
Claude Sonnet 4.6	Direto	91%
Llama 3.1 405B	Ralph Loop	94%
GPT 5.4	Direto	97%

V. Referências

- [1] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Transactions of the Asso-*

ciation for Computational Linguistics, v. 12, pp. 277–294, 2024.

- [2] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022.
- [3] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.
- [4] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan e S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv preprint arXiv:2303.11366*, 2023.
- [5] G. Ghuntley, *Ralph Loop: A Structured Orchestration Pattern for LLM Inference*, Online. Disponível em: <https://ghuntley.com/specs/ralph/>, Acesso em: abr. 2026, 2025.
- [6] S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," *arXiv preprint arXiv:2305.10601*, 2023.
- [7] X. Chen, M. Lin, N. Schärli e D. Zhou, "Teaching Large Language Models to Self-Debug," *arXiv preprint arXiv:2304.05128*, 2023.
- [8] D. Zhou et al., "Least-to-most prompting enables complex reasoning in large language models," *arXiv preprint arXiv:2205.10625*, 2022.
- [9] T. Khot et al., "Decomposed prompting: A modular approach for solving complex tasks," *arXiv preprint arXiv:2210.02406*, 2022.
- [10] L. Wang et al., "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," *arXiv preprint arXiv:2305.04091*, 2023.
- [11] C. McComb et al., "Supervising Ralph Wiggum: Exploring a Metacognitive Co-Regulation Agentic AI Loop for Engineering Design," *arXiv preprint arXiv:2603.24768*, 2026.
- [12] Y. Huang et al., "Beyond English: Evaluating Multilingual Capabilities of Large Language Models," *arXiv preprint arXiv:2401.00000*, 2024.
- [13] F. Shi, M. Suzgun, M. Freitag, X. Wang, M. Surdeanu e J. Wei, "Language Models are Multilingual Chain-of-Thought Reasoners," em *International Conference on Learning Representations*, 2023.
- [14] C. Qin et al., "Cross-lingual Prompting: Enhancing Multilingual Capabilities of Large Language Models," *arXiv preprint arXiv:2302.00000*, 2023.
- [15] X. He et al., "The Impact of Prompt Formatting on Large Language Model Performance," *arXiv preprint arXiv:2403.00000*, 2024.
- [16] Y. Liu et al., "Beyond Prompt Content: Enhancing LLM Performance via Content-Format Integrated Prompt Optimization," *arXiv preprint arXiv:2502.04295*, 2025.

- [17] D. Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” *arXiv preprint arXiv:2311.12022*, 2023.
- [18] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” em *International Conference on Learning Representations*, 2021.
- [19] Y. Wang et al., “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark,” *arXiv preprint arXiv:2406.01574*, 2024.